

Fundación Fulgor

PROCOM

Informe

Sistemas de comunicaciones básico

Autor: Alfici, Facundo Ezequiel

Documento: 44012327

Docentes: Ariel Pola, Santiago Garaventta Pascual y Agustín Pesce

29 de agosto de 2026

Índice

1. Introducción	3
2. Desarrollo	4
2.1. Simulador de Punto flotante	4
2.1.1. Gráficos obtenidos	5
2.2. Simulador de Punto Fijo	8
2.2.1. Gráficos obtenidos	9
2.3. Implentación en Verilog del modelo en punto fijo.	11
2.3.1. Verificación del modelo por Vector Matching	11
2.4. Implementación en FPGA	14
3. Conclusión	17

1. Introducción

En este informe se aplicarán conceptos de los sistemas de comunicación vistos durante las clases, con el fin de diseñar e implementar un sistema de comunicaciones básico, que consta de un PRBS de 9 bits, un filtro RC y un contador de BER.

El trabajo consta de 4 partes;

1. Diseño en Python de un simulador en punto flotante.
2. Diseño en Python de un simulador en punto fijo, utilizando la librería **fixedInt.py** para la cuantización. Teniendo en cuenta además algunos conceptos de diseño de hardware y metodologías de retardo.
3. Implementación en Verilog utilizando punto fijo.
4. Implementación en FPGA.

El diagrama en bloques que representa al sistema completo es el siguiente;

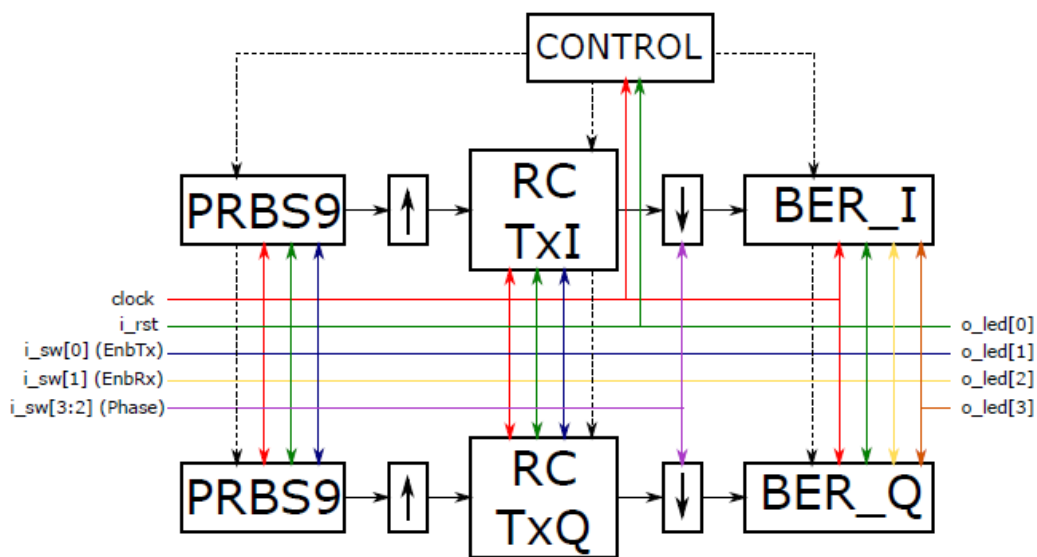


Figura 1.0.1: Diagrama en bloques del sistema de comunicaciones.

2. Desarrollo

2.1. Simulador de Punto flotante

Para este apartado, se pidió implementar un simulador en punto flotante que contemple todo el diseño en donde la representación de la PRBS9 es una secuencia aleatoria y la estimación de la BER es una comparación de vectores. En primera medida, se definió la arquitectura del sistema, de manera que la interconexión entre la PRBS9 y el filtro RC tendrá un upsampling de 4 (dado el oversampling), es decir, por cada muestra que sale de la PRBS9, entrarán 4 muestras al filtro. Ahora, como criterio de diseño, se tomó la decisión de utilizar un sistema de multiplexores que evitan las multiplicaciones por 0 de los coeficientes del filtro con los símbolos enviados por la PRBS.

Este bloque será manejado por un clock lento con período de $1/T_{os}$, de tal manera que, para cada baudio se realizará una selección de la multiplicación a realizar dependiendo si es 0 o 1.

- En caso de que el símbolo sea 0, el coeficiente correspondiente será multiplicado por 1 y enviado al acumulador.
- En caso de que el símbolo sea 1, el coeficiente correspondiente será multiplicado por -1 y enviado al acumulador.

De esta manera, se evitará realizar un mapeado costoso en términos computacionales, método que también se conoce como **filtro polifásico**.

Como consiguiente, el módulo de filtrado contendrá un apartado de graficación, donde se espera ver:

- Bits transmitidos.
- Respuesta al impulso y frecuencia del filtro Tx.
- Salida y diagrama de ojo del filtro Tx.
- Diagrama de constelación a la salida del filtro Tx (por cada fase).

Por otro lado, el contador de BER constará de un slicer previo, donde se busca retomar la cantidad deseada de símbolos reales. Luego, se utilizará una PRBS9 de referencia, que contiene la misma semilla de generación que la PRBS9 de la etapa de transmisión.

Finalmente, es importante destacar los módulos de control y decodificación, donde principalmente se mapea los bits de la entrada `i_sw`, que contiene el enable de transmisión, de recepción y los bits de selección de fase.

2.1.1. Gráficos obtenidos

Lo que respecta a los bits transmitidos por el sistema, se puede ver en la siguiente figura el resultado del tratamiento realizado.

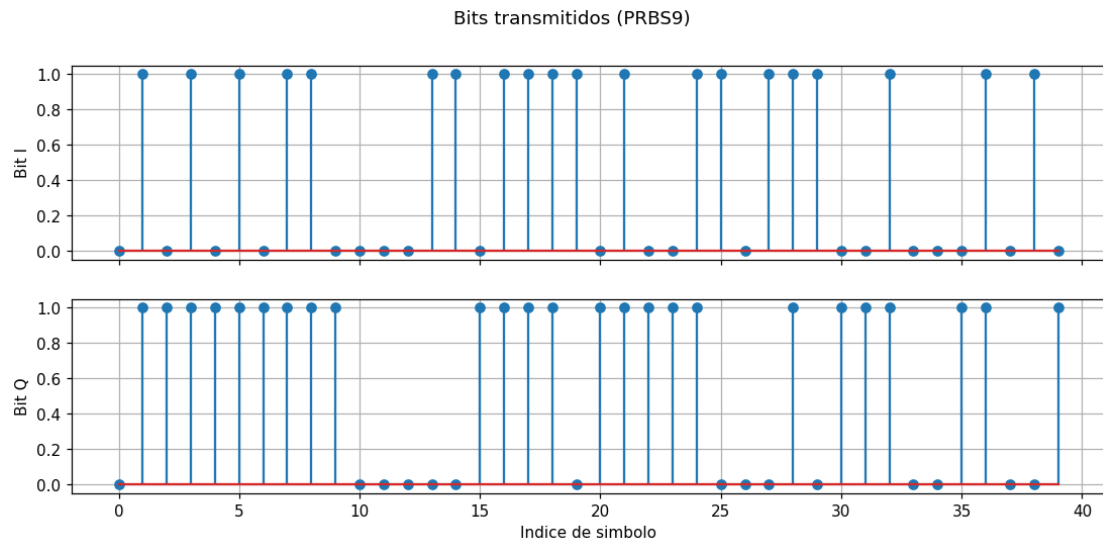


Figura 2.1.1: Bits transmitidos en Fase(I) y Cuadratura(Q) respectivamente.

De igual manera, se puede ver la respuesta al impulso y frecuencia del filtro Tx en la siguiente imagen.

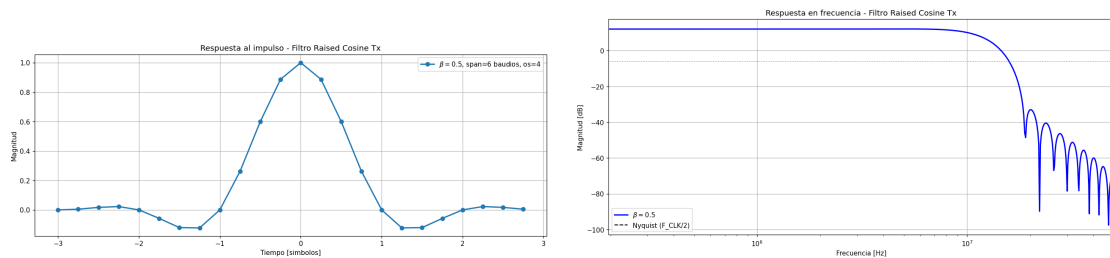


Figura 2.1.2: Respuesta al impulso y frecuencia del filtro Tx.

En la figura 2.1.2 se puede ver la naturaleza del filtro, donde la curva de respuesta al impulso tiene colas muy suavizadas y un valor normalizado a 1, como se pretendía.

En lo que respecta a la salida del filtro polifásico, se tiene la siguiente imagen.

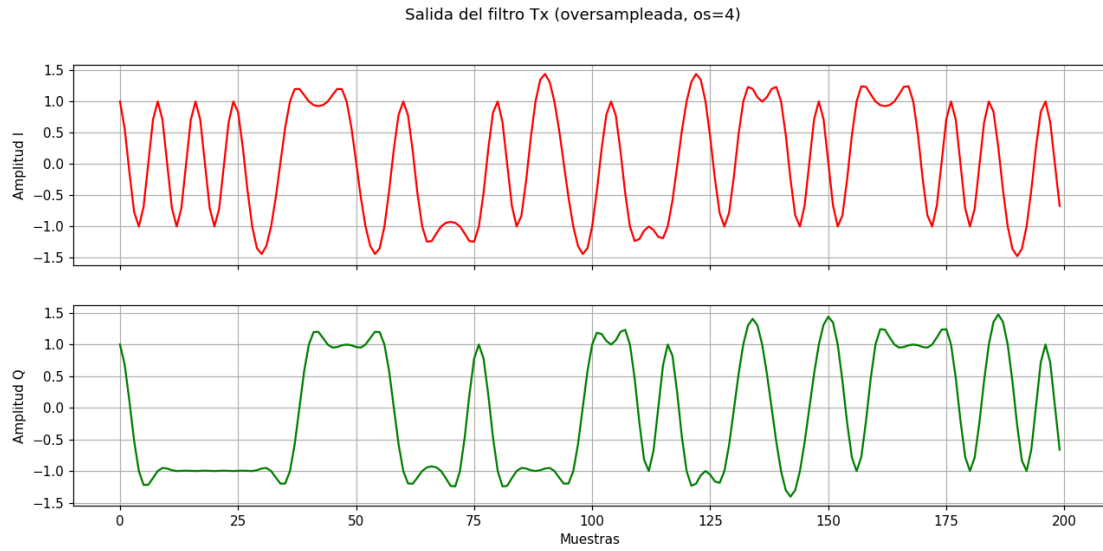


Figura 2.1.3: Salida del filtro polifásico en Fase(I) y Cuadratura(Q) respectivamente.

Esta imagen muestra los bits transmitidos ya filtrados, donde se pueden ver sobrepasamientos esperados en algunos de los símbolos, algo que será clave al momento de elegir la cuantización de los coeficientes del filtro.

Finalmente, el diagrama de ojo y constelación por fase se puede ver a continuación.

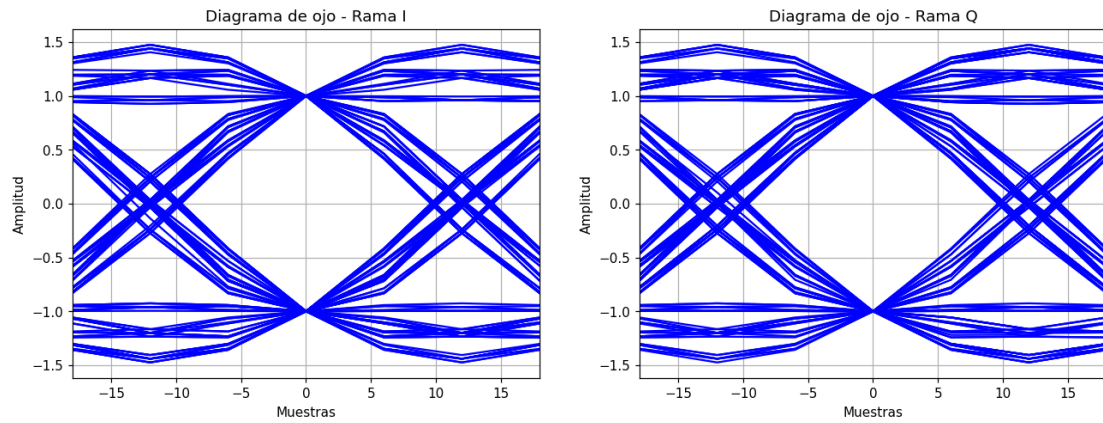


Figura 2.1.4: Diagrama de ojo de fase y cuadratura.

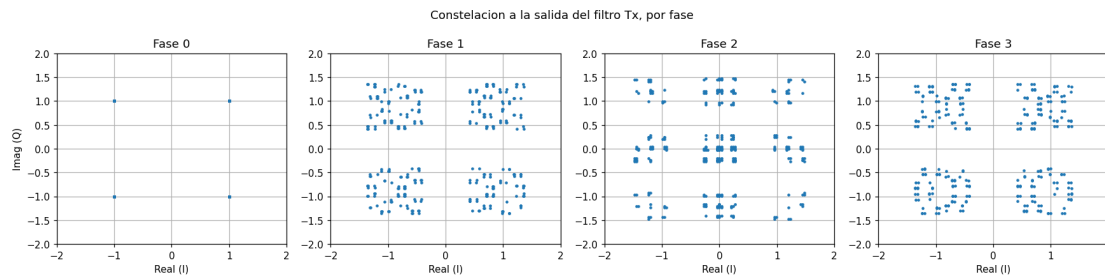


Figura 2.1.5: Constelaciones por fase float simulator.

Estas figuras muestran que, para la fase 0 se tiene una ISI nula, siendo ésta la fase óptima del filtro. Considerando un análisis mas detallado se concluye;

- La fase 0 se representa el símbolo transmitido correctamente.
- Las fases 1 y 3 presentan nubes de puntos alrededor de un punto medio, lo que tiene sentido debido a que son las fases más cercanas a la fase 0.
- La fase 2 tiene errores por el cruce en los ejes del plano complejo.

Este dato se puede corroborar utilizando el contador de BER, donde se muestra el Bit Error Rate para cada fase por separado.

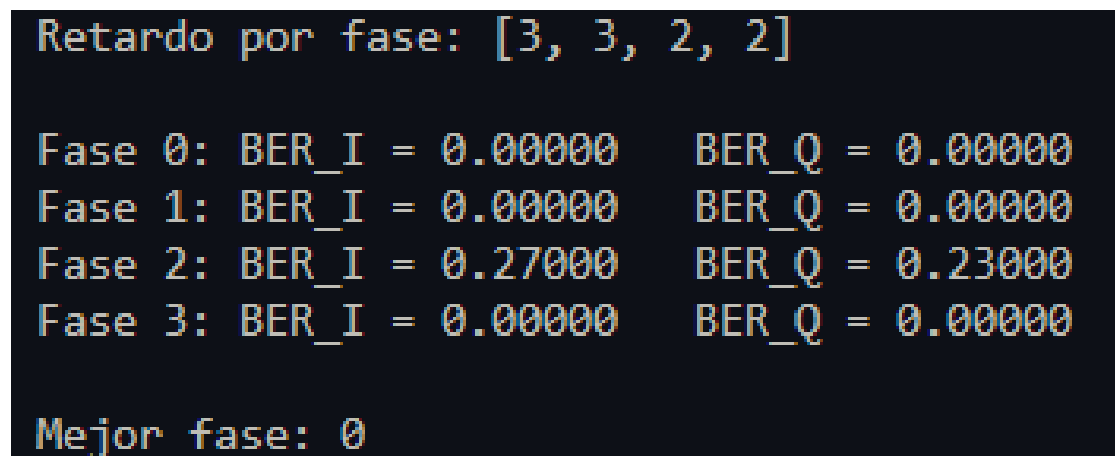


Figura 2.1.6: BER de fase y cuadratura por cada fase.

Algo a destacar es que existirá un retardo por fase introducido manualmente, esto se debe a que el cambio de fase adelanta el generador y genera un corrimiento de la lectura del bit actual, lo que se traduce a un BER elevado para las fases afectadas.

2.2. Simulador de Punto Fijo

Para este inciso se pide realizar algunos cambios de la arquitectura de manera tal que se parezca a una implementación en hardware, agregando también la cuantización en punto fijo.

La selección de la cuantización óptima para este inciso fue $S(9,7)$, debido a que, como ya se comentó antes, la salida del filtro generará picos que sobrepasarán el valor normalizado propuesto para el filtro. Por esta razón, se planteó lo siguiente:

- 1 bit de signo
- 1 bit de parte entera
- 7 bits de parte fraccionaria

De esta manera, se podrá representar cualquier valor que sobrepase el 1 y a la vez tener una buena resolución sin que haya mucha carga computacional. Lo mismo se planteó para la salida de los acumuladores que representan el filtro polifásico, habiendo así un forzamiento a $S(9,7)$ tomando siempre las partes más significativas de las palabras y redondeando el resto.

Lo que respecta al contador de BER, nuevamente se tomó lo planteado en el simulador de punto flotante, donde se utilizó una PRBS9 de referencia para la comparación de los bits recibidos.

Sobre la metodología utilizada para los retardos del sistema, se utilizó una variable que representa el delay por símbolo, de tal manera que el desplazamiento del pico del filtro generado por la cuantización no será significativo.

2.2.1. Gráficos obtenidos

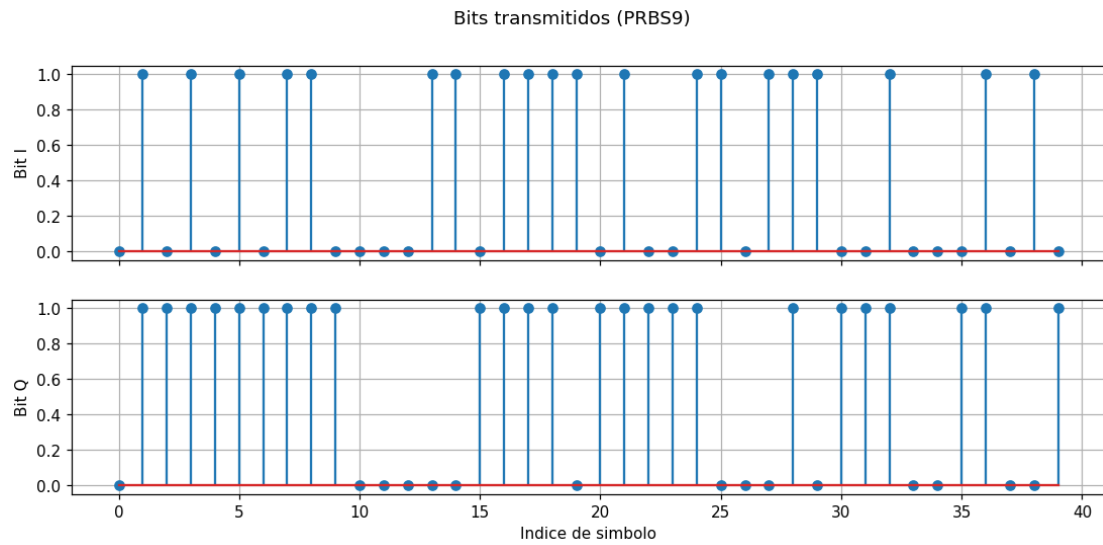


Figura 2.2.1: Bits transmitidos en Fase(I) y Cuadratura(Q) respectivamente.

De igual manera, se puede ver la respuesta al impulso y frecuencia del filtro Tx en la siguiente imagen.

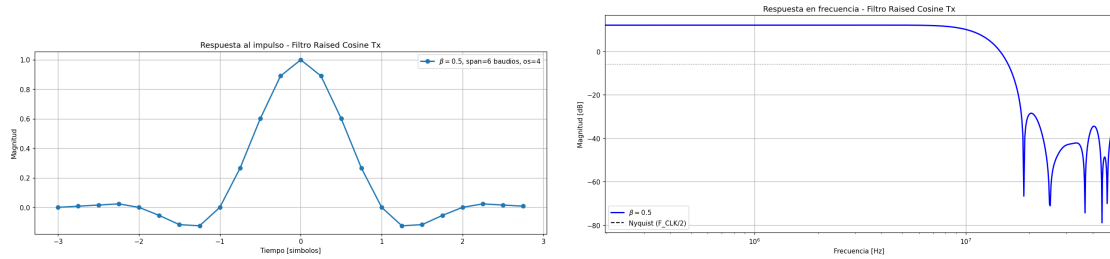


Figura 2.2.2: Respuesta al impulso y frecuencia del filtro Tx con punto fijo.

La respuesta vista en la figura 2.2.2 nos dice que la representación de la salida no se verá afectada en mayor medida debido a la cuantización, por lo que podríamos esperar un resultado muy similar al obtenido en el modelo de punto flotante para los demás apartados.

En lo que respecta a la salida del filtro polifásico, se tiene la siguiente imagen.

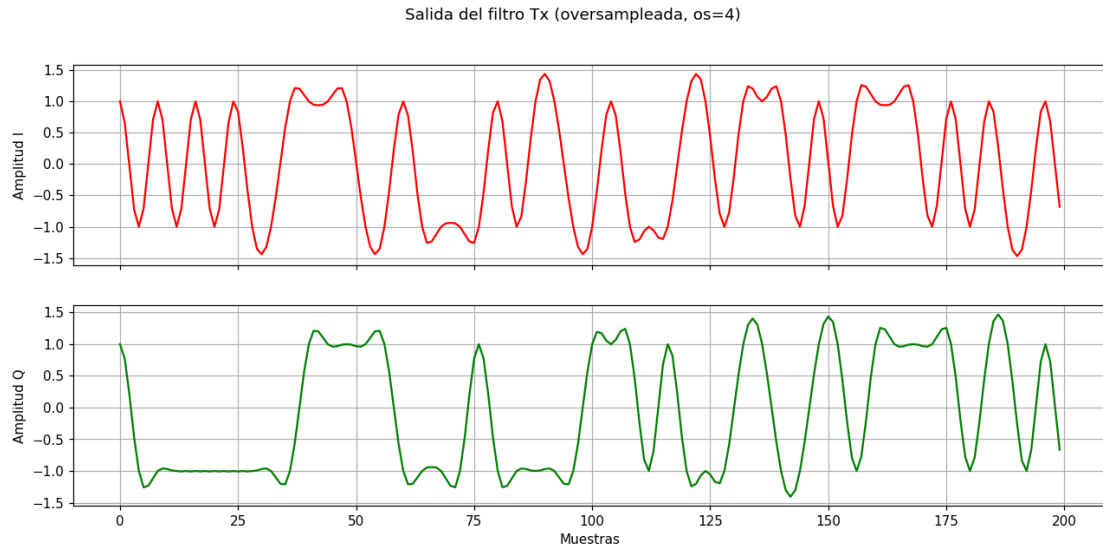


Figura 2.2.3: Salida del filtro polifásico en Fase(I) y Cuadratura(Q) en punto fijo.

Comparado el resultado obtenido con la figura 2.1.3 se puede ver que la representación de los símbolos es muy fiel.

Finalmente, el diagrama de ojo y constelación por fase se puede ver a continuación.

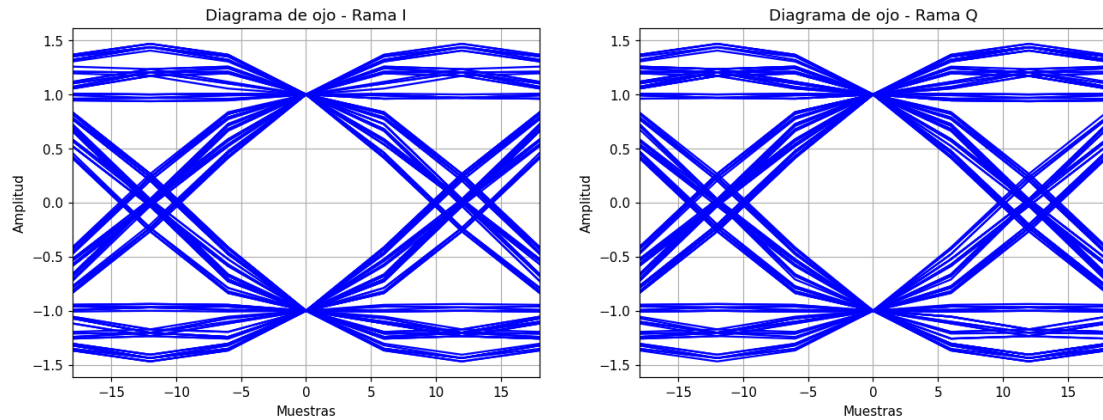


Figura 2.2.4: Diagrama de ojo de fase y cuadratura en punto flotante.

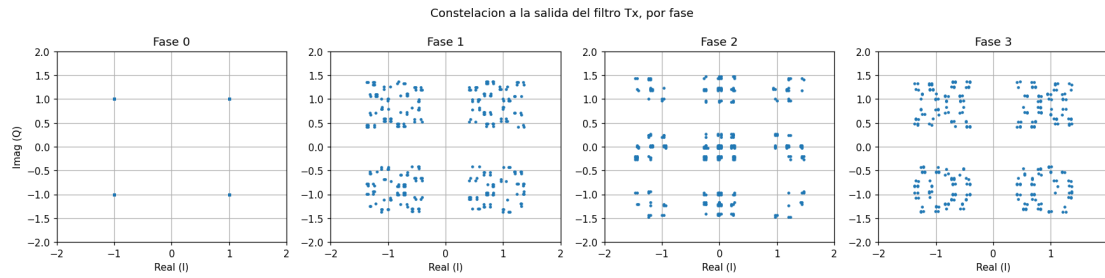


Figura 2.2.5: Constelaciones por fase fixed point simulator.

Lo visto demuestra que, el simulador de punto fijo con arquitectura de hardware real se comporta correctamente, dando resultados prácticamente iguales a los obtenidos en el simulador de punto flotante, como se muestra en la figura 2.1.5

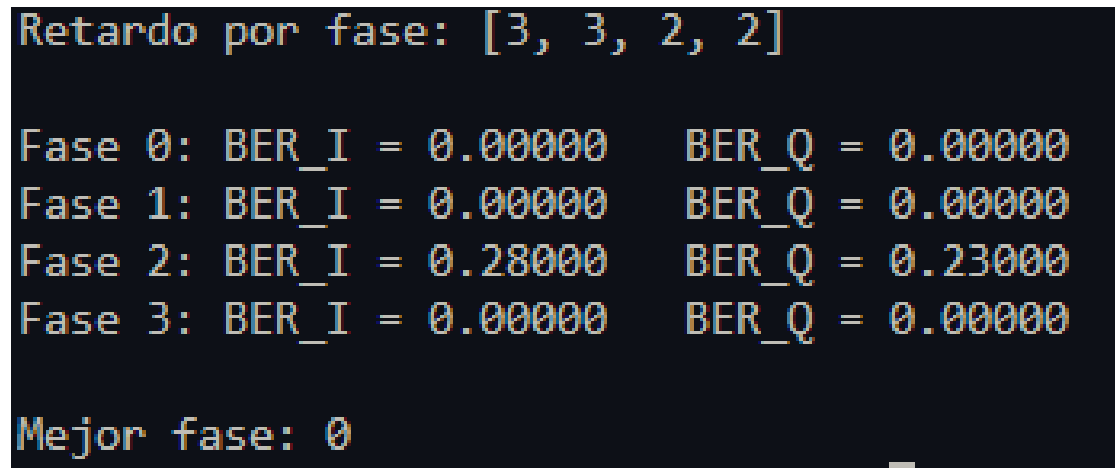


Figura 2.2.6: BER de fase y cuadratura por cada fase de punto flotante.

2.3. Implentación en Verilog del modelo en punto fijo.

En este inciso, se desarrollará el modelo en punto fijo en Verilog, previamente propuesto en Python. Siendo un lenguaje donde se pretende desarrollar el programa de la forma más prolija posible, se dividió el código en bloques funcionales para cada etapa.

2.3.1. Verificación del modelo por Vector Matching

Para verificar el modelo, se realizó un testbench que utiliza los estímulos realizados y las salidas obtenidas del modelo realizado en punto fijo en Python, con el fin de comparar con los resultados obtenidos en este modelo en Verilog. Esto se logró

utilizando un archivo **.mem** generado dentro del modelo en Python, cargándolo luego en el testbench por medio de la función **\$readmemb** lo que permitió utilizar los valores para compararlos uno por uno durante toda la simulación del top_level.

Los waveforms obtenidos se pueden ver a continuación, considerando el análisis para cada una de las fases.

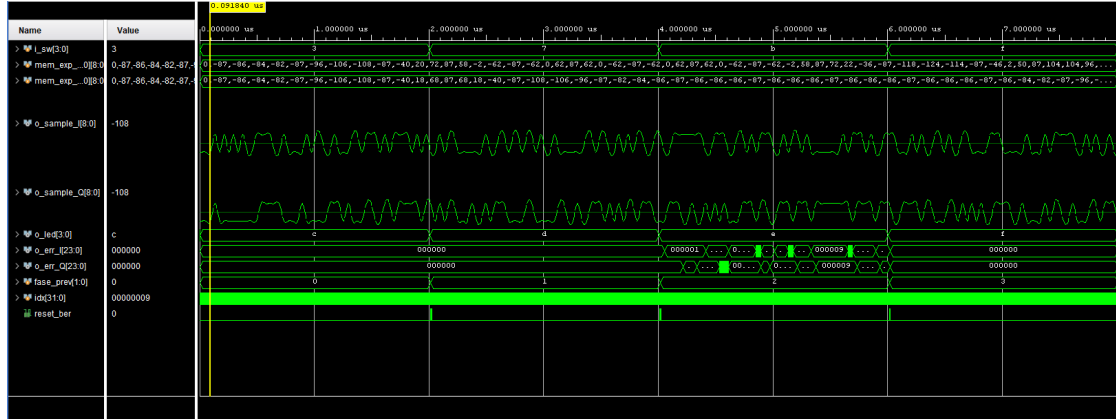


Figura 2.3.1: Waveforms para todas las fases

Estas formas de onda permiten ver todas las fases en su totalidad, realizando cambios de fase cada 2uS, de tal manera que se puede observar la fase completa de esta forma.

Como se vió en la figura 2.2.6, el BER para las fases 0, 1 y 3 son nulas, mientras que la fase 2 es la que contiene un error en los bits transmitidos.

Esto se puede ver en mejor medida realizando zooms en cada una de las fases, como se puede ver a continuación.

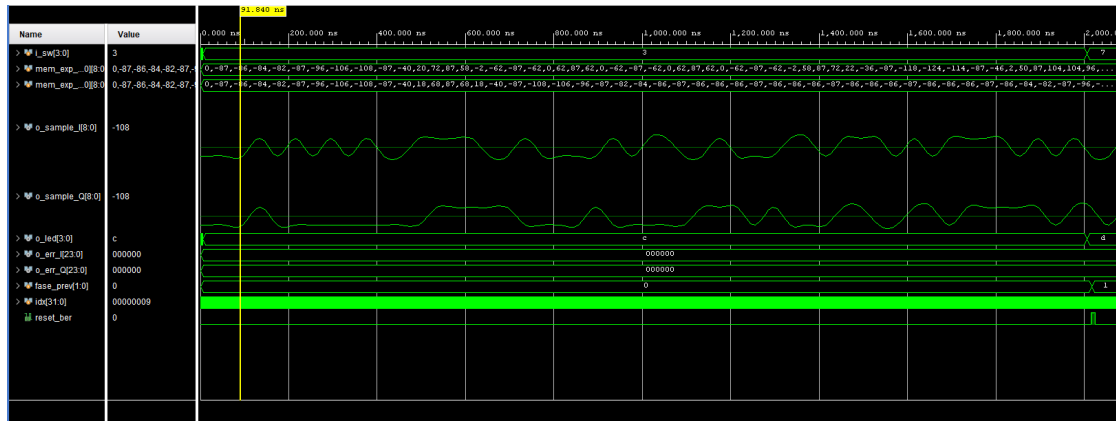


Figura 2.3.2: Waveforms de la fase 0

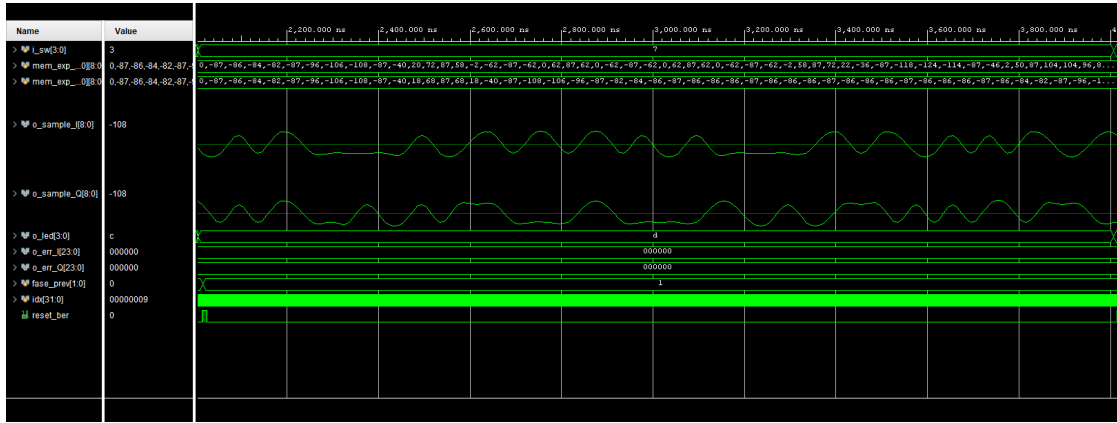


Figura 2.3.3: Waveforms de la fase 1

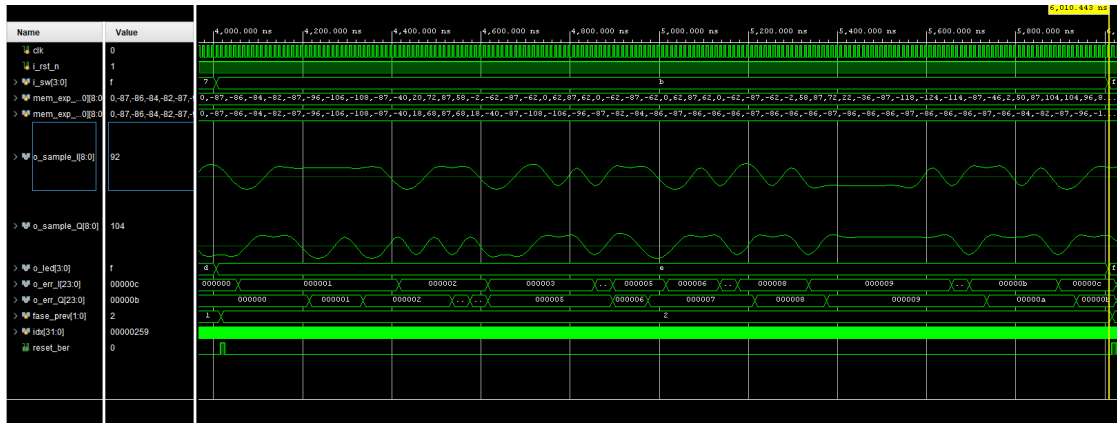


Figura 2.3.4: Waveforms de la fase 2

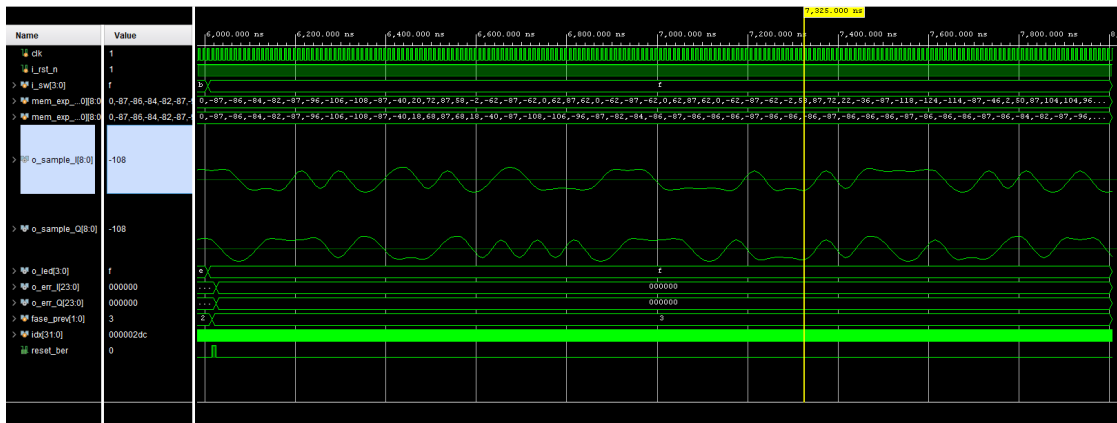


Figura 2.3.5: Waveforms de la fase 3

Para cada fase, el Bit Error Rate dividido en fase y cuadratura se puede ver en las variables `o_err_I` y `o_err_Q`.

Finalmente, se muestra en la consola la comparación entre los valores obtenidos en la implementación y en el modelo de Python, siendo los siguientes.

```
BER fase 0: I = 0/47  Q = 0/47
BER fase 1: I = 0/50  Q = 0/50
BER fase 2: I = 12/50  Q = 11/50
BER fase 3: I = 0/49  Q = 0/49
Vector Matching finalizado: 2 mismatches de 801 ciclos
$finish called at time : 8006 ns : File "C:/Users/facun/VSCoDeProjects/PROCOM/Lab4/tb_top_level.v" Line 69
```

Figura 2.3.6: Mismatches de la implementación, con BER por fase en relación a la cantidad de símbolos

Los mismatches representan una diferencia entre el valor obtenido por símbolo y el valor del `.mem`, estos 2 errores pueden ocurrirse debido a un desfase de alguna de las mediciones obtenidas.

2.4. Implementación en FPGA

Este apartado consistirá en mostrar los resultados obtenidos al momento de realizar la carga del programa dentro de una FPGA de la fundación.

En primera medida, es necesario recalcar que se utilizó los IP Cores VIO e ILA para manejar y visualizar algunos de los parámetros necesarios para el funcionamiento del sistema de comunicaciones. Las variables a controlar serán:

- `i_sw`: Manejando la habilitación de la transmisión, recepción y la selección de fases.
- `i_reset`: Manejando el reset de la placa para habilitar el funcionamiento.

Mientras que las variables importantes a visualizar serán:

- `o_sample_I`: Representa la salida del filtro en Fase.
- `o_sample_Q`: Representa la salida del filtro en Cuadratura.
- `o_led`: Representa a los leds que muestran el estado actual de `i_sw`.
- `o_err_I`: Representa el BER para Fase.
- `o_err_Q`: Representa el BER para Cuadratura.
- `o_ber_ok`: Led que se enciende al tener BER igual a 0.

Finalmente, se obtuvieron las siguientes formas de onda para cada fase, utilizando un trigger configurado para que se dispare cuando el led de BER igual a 0 se encienda.



Figura 2.4.1: Waveforms de la fase 0 en Hardware

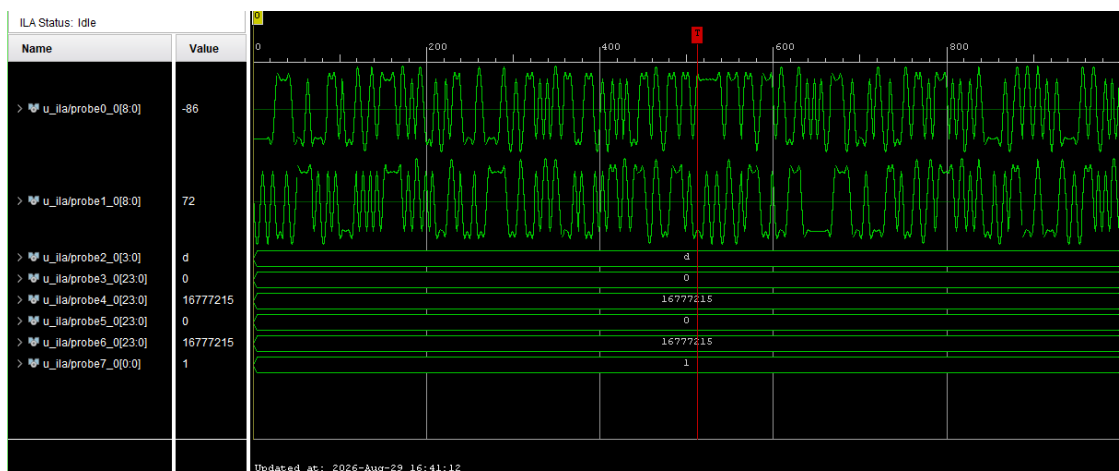


Figura 2.4.2: Waveforms de la fase 1 en Hardware

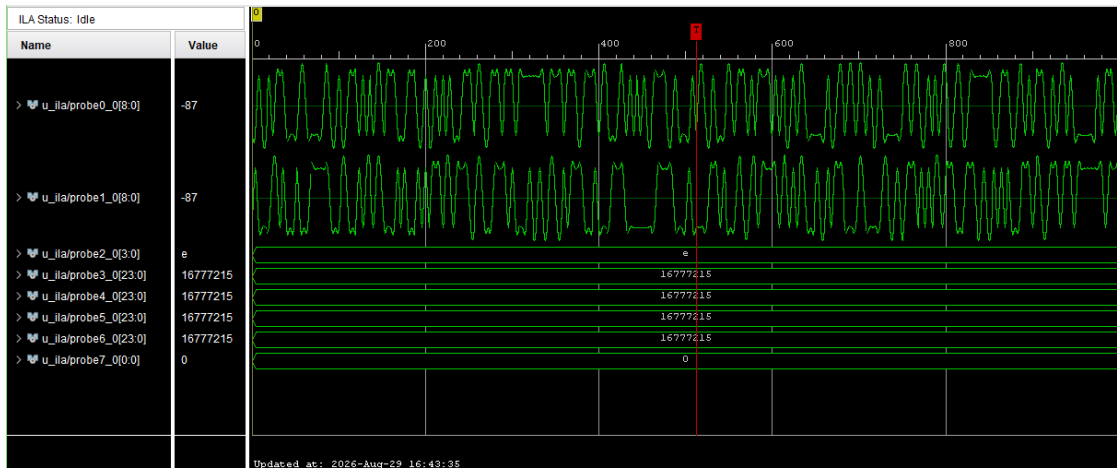


Figura 2.4.3: Waveforms de la fase 2 en Hardware

Para esta fase se realizó un cambio en el trigger, dado que para la misma la BER era distinta de cero en todos los casos, el trigger nunca se disparaba al esperar que el led se encienda, entoces se configuró de tal manera que se dispara cuando el led estaba apagado.

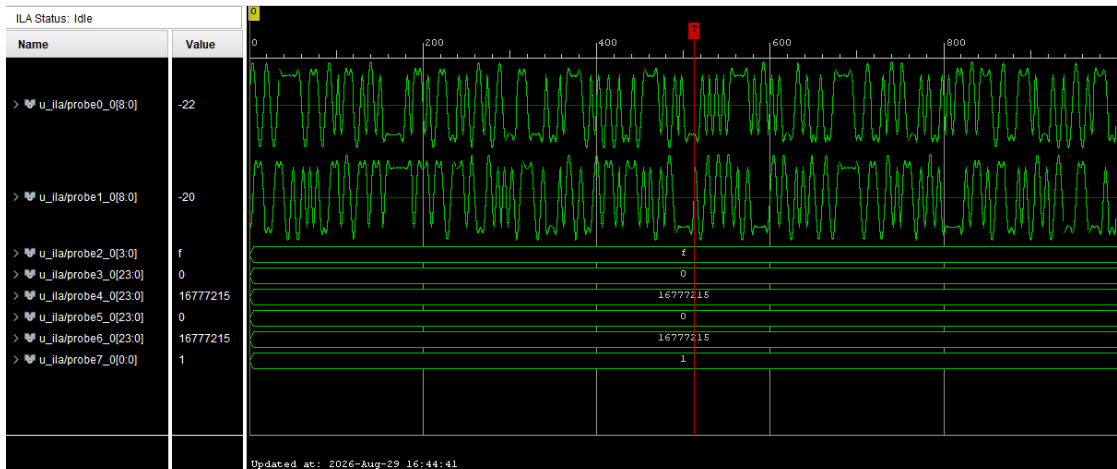


Figura 2.4.4: Waveforms de la fase 3 en Hardware

3. Conclusión

Se llegó a obtener el resultado esperado para la implementación en Hardware, por lo que se concluye un trabajo exitoso. Es destacable la utilidad obtenida mediante la simulación, dado que la implementación mostró igualdad 1 a 1 con lo obtenido en Python.

La única diferencia obtenida fue que, el contador de BER de la fase 2 se saturaba luego de mostrar la fase pues, el reset del contador del Bit Error Rate se activaba cuando se cambia de fase, y al no cambiar de fase se seguían acumulando las muestras, cosa que no ocurría en la simulación pues ésta solamente llegaba al valor máximo de representación para 1 vuelta por cada fase. Esto se puede solucionar mediante el cambio de la condición de reset del contador, pero para mantener la fidelidad con el simulador se prefirió dejarlo en su estado actual.